Syntapse: A Memory-First, Multi-Agent Platform for AI-Assisted Software Engineering

T Mohamed Yaser Mohammed Abubakar Mohammed Sabeehuddin Abdul Rafe

***Abstract*— AI-assisted software engineering tools remain bound to a single conversation or file context: chat-based assistants forget earlier design decisions, prompt-chained agent frameworks pass large blocks of text from one step to the next so that one failure breaks the chain, and founders and developers still spend long hours drafting product requirement documents (PRDs) and technology stacks by hand. This paper presents *Syntapse*, a memory-first, multi-agent platform for AI-assisted software engineering in which a shared persistent graph memory, rather than the prompt, is the single source of truth. Specialized agents run as background daemons that never communicate directly: each observes memory mutations announced on an event bus, evaluates whether its own work product is missing or invalid, acts by retrieving context and invoking a language model, commits the result back to the graph, and returns to sleep. When a developer edits a memory node on an interactive canvas, the resulting event causes dependent nodes to be regenerated, which we call a self-healing event loop. Syntapse is implemented with Cognee and Memgraph for graph memory, Redis Pub/Sub for events, a Python FastAPI backend, a Next.js canvas interface, and local (Ollama, Qwen2.5-Coder) or hosted (Gemini) models. We position the platform against recent shared-state, memory and persistent-orchestration systems, formalize the context-fragmentation problem, describe the architecture, memory model, event mechanism and human-in-the-loop workflow, and give an evaluation methodology. The contributions are design contributions; no experimental improvement is claimed.**

Index Terms— software engineering, large language models, multi-agent systems, graph memory, event-driven architecture, human-in-the-loop, software development life cycle

I Introduction

Software engineering is inherently collaborative and highly contextual. A project accumulates requirements, design decisions, rejected alternatives and code over weeks or months, and the value of any new contribution depends on how well it is consistent with that accumulated knowledge. When this activity is automated with large language models (LLMs), the models are typically forced to work in isolation: text is passed from one model call to the next in the hope that critical requirements are neither forgotten nor hallucinated.

I.1 Failure Modes of Sequential AI Orchestration

Three failure modes motivate this work. **Context loss:** LLMs forget early design decisions, which makes macro-level orchestration of a whole project impractical. **Sequential failure:** agent frameworks based on sequential prompt chaining pass the output of agent A to agent B and then to agent C; when requirements change, or when one agent makes a mistake, the chain breaks, the context is lost, and the usual remedy is to re-run everything from scratch, because context is treated as a temporary payload rather than as a permanent state of truth. **Manual overhead:** even with AI assistance, founders and developers still spend a large amount of time drafting PRDs and choosing technology stacks before any code is written.

I.2 Problem Statement

The problem addressed in this paper is how to give a team of AI agents a *durable, shared and structured* understanding of a software project so that (i) knowledge persists across sessions, (ii) all agents rely on the same knowledge, (iii) a change to upstream knowledge automatically causes the affected downstream work to be reconsidered, and (iv) the developer retains control by inspecting and editing that knowledge.

I.3 Approach

Syntapse is a *memory-first* platform. Instead of passing prompts between agents, every specialist reads from and writes to the same persistent memory graph. When one agent finishes a work product, it commits that knowledge to long-term memory; a downstream agent observes the memory mutation, wakes up, recalls the exact context it needs, and begins its own work. The agents do not communicate with each other directly.

I.4 Contributions

1. A **memory-first architecture** in which all agents share one persistent graph memory (Cognee over Memgraph) that serves as the project's source of truth.
2. An **event-driven agent lifecycle** (Observe, Evaluate, Act, Sleep) in which memory mutations announced on a Redis event bus trigger downstream reasoning without sequential prompt chaining.
3. A **change-triggered, self-healing update mechanism**: when a memory node is edited, dependent nodes are discarded and regenerated from the updated state.
4. A **memory canvas** through which the developer sees agent activity as a timeline of memory events

and visually edits memory nodes, together with an **implementation-strategy workflow** in which the developer builds with the platform's guidance and submits the result for review.

5. A **formal reference model and evaluation methodology** for assessing context preservation and coordination in memory-based multi-agent engineering systems.

I.5 Scope of Claims

The contributions are design contributions. No experimental results are reported, and the paper does not claim that Syntapse resolves limitations such as LLM reliability, cost or scale. Statements about expected benefits are design intentions to be tested by the methodology of Section XIII.

I.6 Organization

Section II reviews related work and the research gap. Section III formalizes the problem. Sections IV to X present the architecture, memory model, event mechanism, agents, workflow, context assembly and user interface. Section XI describes the implementation. Sections XII to XV discuss design trade-offs, the evaluation methodology, security and ethics, and limitations. Section XVI outlines future work and Section XVII concludes.

II Related Work and Research Gap

II.1 Role-Based Multi-Agent Software Engineering

ChatDev organizes software development as a communication workflow among role-based AI agents that cover design, coding and testing [1]. MetaGPT organizes agents around software-company roles with structured outputs and defined workflows covering requirements analysis, design and implementation [2]. A survey of agents in software engineering reviews the capabilities, architectures and open challenges of this area and frames agents in terms of perception, memory and action modules [3]. These works show that role specialization and standardized artifacts help agents collaborate. They coordinate through conversational exchanges or staged workflows, whereas Syntapse coordinates through a shared persistent graph that agents read and update asynchronously.

II.2 Memory Mechanisms for LLM Agents

Surveys of agent memory examine how language-model agents store, retrieve, update and use information over time [4], and formalize memory as a write–manage–read loop together with evaluation benchmarks for multi-session agentic tasks [5]. These bodies of work are relevant to the context-fragmentation problem because long-running software projects must preserve requirements and design decisions between interactions. Their scope is general; Syntapse applies persistent graph memory specifically to the software development life cycle.

II.3 Shared State and Blackboards for Software Agents

SE-Blackboard compares message passing, a shared-state blackboard and a hybrid in a Planner–Coder–Reviewer–Tester bug-fixing pipeline [6]. As reported in that work, the blackboard keeps the original issue visible to every agent and raises Coder-stage information fidelity (0.586 versus 0.362) and correct-file targeting (82.6% versus 55.1%) on 50 SWE-bench Lite issues with one LLM, while the resolve-rate gain (16% versus 12%) was not statistically significant, the blackboard used about 2.2 times the tokens, and the hybrid produced empty patches in 36% of runs. The lesson drawn is that memory should be surfaced selectively and not all at once. The study covers bug fixing only and offers no notification when shared state changes.

II.4 Event-Based Distributed Memory

MemIndex manages memory for LLM-based publish/subscribe agent systems using intent-indexed bipartite graphs grouped into semantic slices, with negotiation between agents and an optimizer subagent that deletes outdated entries and offloads overloaded slices [7]. It is reported to reduce elapsed time, CPU use and memory use relative to tabular and Neo4j baselines in a smart-building scenario with synthetic text-only data; accuracy depends on the LLM. It manages memory itself and does not manage software-development work products.

II.5 Persistent Orchestration

TheBotCompany drives milestones through a Strategy, Execution and Verification state machine in which manager agents hire and retire workers, agents communicate through a SQLite issue tracker, and humans oversee asynchronously [8]. On four long-running projects it completed 164 milestones, and on five ProjDevBench problems it reported a mean execution score of 53.2 against 43.6 for Claude Code with the same model at about 1.65 times the output tokens per attempt. The results are single runs with one model family. Mid-project requirement steering is left as future work, and a bounded context window is reported to be unable to track accumulated knowledge over many milestones, which motivates keeping long-term project knowledge in persistent storage outside the agents' context.

II.6 Other Shared-Memory Coordination

The Shared Recurrent Memory Transformer (SRMT) lets decentralized agents write recurrent memory states to a shared workspace that others read by cross-attention [9]. It targets multi-agent pathfinding, assumes grid worlds and holds neural, not human-readable, memory, so it is relevant only

Table 1: Existing Systems and Relevant Limitations

System	Main approach	Relevant limitation
SE-Blackboard [6]	Shared-state blackboard vs. message passing	(A) Small non-significant resolve gain; hybrid gave 36% empty patches. (O) No change notification
MemIndex [7]	Intent-indexed graph memory with agent negotiation	(A) Accuracy varies by LLM. (O) Manages memory, not SDLC work products
TheBotCompany [8]	Milestone cycle with hired and retired workers	(A) Single-run; mid-project requirement steering left as future work
SRMT [9]	Shared recurrent memory read by attention	(A) Pathfinding only. (O) Neural memory, not readable project knowledge
Society Agent [10]	Persistent supervisors, ephemeral workers, file memory	(A) File I/O limit near 50 agents. (O) No relationship graph

Table 2: Categories of AI Engineering Tools (Qualitative)

System	Interaction	Context memory	Collaboration	Stability
Chatbots (e.g., ChatGPT)	Manual prompting	Short-term (session)	None	High, but limited scope
Agent frameworks (e.g., AutoGPT, Devin)	Sequential chaining	Flat text passing	Linear, agent-to-agent	Lower: loop and hallucination risk
Syntapse (proposed)	Graph-based state	Persistent graph DB	Shared state	Intended: self-correcting loop

as a mechanism of shared-memory coordination. Society Agent combines persistent supervisors, ephemeral workers, file-based memory, a message queue, scheduling and heartbeat monitoring [10]; it is a preprint whose file I/O is reported to become a bottleneck beyond about 50 concurrent agents, and it has no relationship graph.

II.7 Summary of Limitations of Existing Work

Table 1 summarizes the main approach of each system and the limitation relevant to this work. Limitations marked (A) are acknowledged by the authors of the respective papers; those marked (O) are our own observations from the comparison.

II.8 Positioning Against Tool Categories

Table 2 characterizes the main categories of AI engineering tools along interaction model, context memory, collaboration style and stability. The characterization is qualitative and concerns typical design, not measured performance.

II.9 Research Gap and Research Question

The reviewed work shows that shared state, event-based memory management and persistent orchestration have each been explored, but separately and in different settings. SE-Blackboard keeps per-issue state for a bug-fixing pipeline, MemIndex combines graph memory with publish/subscribe outside software engineering, and TheBotCompany persists project state in an issue tracker with orchestrator-mediated phases. None of the three examines software artefacts across the life cycle (requirements, design, code and tests) held in a persistent graph whose changes are broadcast as events to the agents that depend on them. We present this as an opportunity for investigation and as a *potential* gap, not as a claim of absolute novelty, since a systematic literature search would be needed to confirm that no comparable system exists. The research question is:

How can persistent graph-based memory and event-driven communication improve context preservation and coordination among autonomous AI agents throughout the software development life cycle?

Whether the design improves context preservation and coordination remains an open question to be tested; no improvement is assumed.

III Problem Formulation

III.1 Definitions

Let a project be developed over a sequence of sessions $s_1, \ldots, s_n$. Let K_t denote the set of knowledge items about the project (requirements, decisions, code changes, test outcomes, and so on) that exist at time t. An assistant answering a request q at time t is given a context $C(q,t) \subseteq K_t$ drawn under a size budget B, i.e., $|C(q,t)| \leq B$.

Context fragmentation occurs when items that are relevant to q are absent from $C(q,t)$. Define the relevant set $R(q,t) \subseteq K_t$ and the *context recall*

$$\rho(q,t) = \frac{|C(q,t) \cap R(q,t)|}{|R(q,t)|}. \tag{1}$$

In a prompt-bound assistant, $C(q,t)$ contains only what the developer supplies in the current session, so ρ typically falls as the project grows and as time since an item was created increases. A memory-first assistant instead computes $C(q,t) = \mathrm{retrieve}(M_t, q, B)$ from a persistent memory M_t that accumulates K_t, so that ρ depends on retrieval quality and not on whether the developer remembers to restate an item.

III.2 Design Goals

From the problem statement we derive the following goals.

G1 Persistence. Project knowledge shall survive across sessions without being re-entered.

Table 3: Forms of Context Loss and Corresponding Mechanism

Form	Description	Mechanism
Session loss	Knowledge from earlier sessions is unavailable	Persistent memory (G1)
Role divergence	Different tools or agents hold different views	Single shared memory (G2)
Rationale loss	The reason for a decision is forgotten	Relations and history (G4)
Propagation lag	A change is not reflected in dependent work	Events (G3)

G2 Shared view. All lifecycle roles shall read and write the same memory, so no role has a private, divergent copy of project knowledge.

G3 Propagation. A change to project knowledge shall automatically trigger the reasoning that depends on it.

G4 Structure. Knowledge shall be stored with explicit relations so that dependencies and rationale can be traced.

G5 Human control. The developer shall decide what is integrated into the codebase, and verification shall be an explicit, repeatable step.

G6 Transparency. The accumulated knowledge shall be inspectable by the team.

G7 Model independence. Memory shall be independent of the particular model used, allowing local and hosted models to be interchanged.

III.3 Taxonomy of Context Loss

Table 3 distinguishes four forms of context loss that the design goals are meant to address. Separating them clarifies which mechanism of Syntapse responds to which problem.

Session loss is the most visible form and is what developers experience as having to re-explain the project. *Role divergence* arises when, for example, a planning tool and a coding tool are configured separately and each holds a different version of the requirements. *Rationale loss* is subtle: the code remains but the reason it looks as it does is gone, so later changes may unknowingly undo it. *Propagation lag* occurs when a requirement changes but the design, code and tests that depend on it are not revisited until a defect appears.

III.4 Scope and Non-Goals

Syntapse is not designed to replace the developer or to autonomously push changes to a repository. It does not claim formal correctness guarantees for generated code; verification is a model-assisted review whose outcome the developer interprets.

IV System Architecture

IV.1 Overview

Syntapse is organized as a memory-first operating layer for AI software engineering (Fig. 1). The developer interacts through one web interface. Behind it, five layers cooperate: a presentation layer, an orchestration layer containing the specialized agents, an event layer, a memory layer, and an inference layer.

IV.2 Layers

Presentation layer. The Next.js "OS canvas" accepts the initial project prompt, renders the memory graph and a timeline of memory events, lets the developer edit memory nodes, and provides the build-strategy view in which the developer submits the result of implementation work for review. State and timeline updates reach the interface over WebSockets.

Orchestration layer. Five specialized agents run as background daemons, completely decoupled from the user interface. Each is a behaviour (an evaluation rule and a prompt) for one engineering activity. The backend is written in Python with FastAPI and asyncio.

Event layer. A Redis Pub/Sub event bus carries a graph-mutated event from the memory layer to every subscribed daemon, which is the mechanism for propagation (G3).

Memory layer. Cognee manages the memory graph, with Memgraph as the underlying graph database. This layer is the single system of record (G1, G2, G4) and is referred to in the project as "the brain".

Inference layer. Local inference uses Qwen2.5-Coder served through Ollama, and hosted inference uses Gemini. The inference layer is stateless with respect to the project; all project knowledge comes from memory (G7).

IV.3 Specialized Agents

Syntapse deploys five specialized, autonomous agents, each a daemon responsible for one engineering activity (Table 4). None of these agents communicates directly with another. Each observes the shared memory graph: when the PRD agent finishes writing requirements, it commits that knowledge to long-term memory; the downstream System Architect observes this memory mutation, wakes up, recalls the exact context it needs, and begins its work. This removes the dependence on a chain of messages and thereby the possibility that one failed handoff breaks all later steps.

IV.4 Technology Stack

Table 5 lists the components and their responsibilities. The project's rationale for Cognee is that standard vector databases cannot represent the hierarchical, relational constraints of software engineering, while standard graph databases lack the semantic search needed to retrieve context for AI agents; Cognee is used as a memory graph that understands semantic meaning and stores relationships.

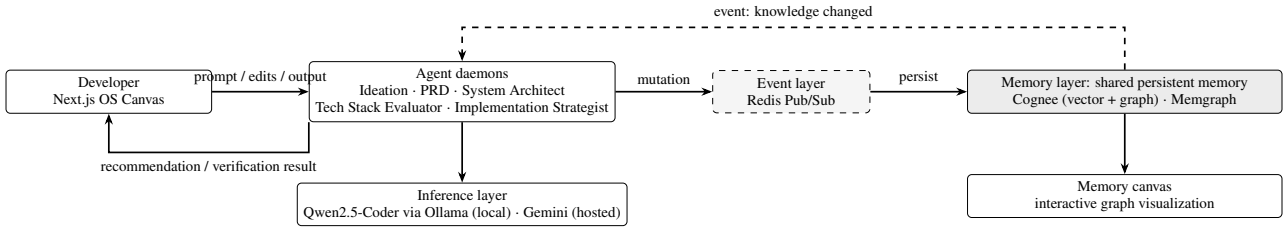


Figure 1: High-level architecture of Syntapse. All roles read and write one shared memory; memory mutations are published on the event bus and trigger downstream reasoning. The developer remains in the loop.

Table 4: Agent Daemons and the Memory Nodes They Produce

Agent	Stage	Node on the canvas
Ideation Specialist	Ideation	Product Vision (from the Raw Prompt seed)
Product (PRD) Architect	PRD	Product Requirements
System Architect	Architecture	System Design
Tech Stack Evaluator	Tech stack	Engineering Stack
Implementation Strategist	Implementation	Build Strategy

Table 5: Technology Stack and Responsibilities

Component	Responsibility
Cognee	Semantic memory graph: ingestion, relationships, retrieval
Memgraph	Graph database underlying the memory graph
Redis (Pub/Sub)	Event bus for graph-mutated events
Python, FastAPI, asyncio	Backend services and agent daemons
Next.js (React), framer-motion, react-force-graph	Frontend OS canvas, animation and 3D graph rendering
Supabase	User authentication and basic persistence
Ollama (Qwen2.5-Coder 3B)	Local model inference
Gemini API	Hosted model inference
Docker Compose	Local deployment of Redis and Memgraph

IV.5 Interface Contracts Between Layers

Table 6 states what each layer offers to the layers above it. Keeping these contracts narrow is what allows layers to be replaced independently, for example substituting one model for another or one graph store for another.

IV.6 Design Rationale

Why memory as the coordination layer? If roles exchange prompts, the shared understanding is implicit in message history and is lost when the conversation ends. If roles exchange knowledge through memory, the shared understanding is explicit, persistent and inspectable. *Why events?* Without events, someone (the developer or a controller script) must decide which role to invoke after each change. Events make the dependency of reasoning on knowledge explicit: a role subscribes to the kinds of change that concern it. *Why a graph in addition to vectors?* Vector similarity retrieves text that is semantically close to a query but does not encode that a component *depends on* an interface or that a decision *supersedes* an earlier one. Graph relations carry that structure.

Table 6: Interface Contracts Between Layers

Layer	Offers	Requires
Presentation	Task entry, submission, canvas	Recommendations, results, graph data
Orchestration	Role behaviours	Context, inference, event delivery
Event	Publish and subscribe	Mutation records
Memory	Ingest, link, supersede, retrieve, archive	Items with provenance
Inference	Text generation from a prompt	Prompt with role, context, request

V Memory Model

V.1 Stage-Level Structure

At the coarsest level, the memory graph is organized around the stages through which a project idea matures: *Ideation*, *PRD*, *Architecture*, *Tech Stack* and *Implementation*. Each stage is a family of nodes in the shared graph; agents read the nodes of earlier stages and write or regenerate the nodes of the stage they are responsible for (Fig. 2). Because every stage reads and writes the same graph, the architecture stage sees the PRD and the idea, the tech-stack stage sees the architecture, and the implementation stage sees all of them.

V.2 Fine-Grained Reference Schema

Within and across stages, we describe project memory with the following reference model; it is the conceptual vocabulary used in this paper and refines the stage-level view. Memory is a typed property graph $M = (V, E)$. Vertices V represent knowledge items and edges E represent relations.

Typical relations include *satisfies* (Component $\rightarrow$ Requirement), *depends-on* (Component $\rightarrow$ Component or Interface), *realizes* (CodeChange $\rightarrow$ Component), *verified-by* (CodeChange $\rightarrow$ Verification), *supersedes* (Decision $\rightarrow$ Decision) and *motivated-by* (Decision $\rightarrow$ Requirement). Fig. 3 illustrates the schema.

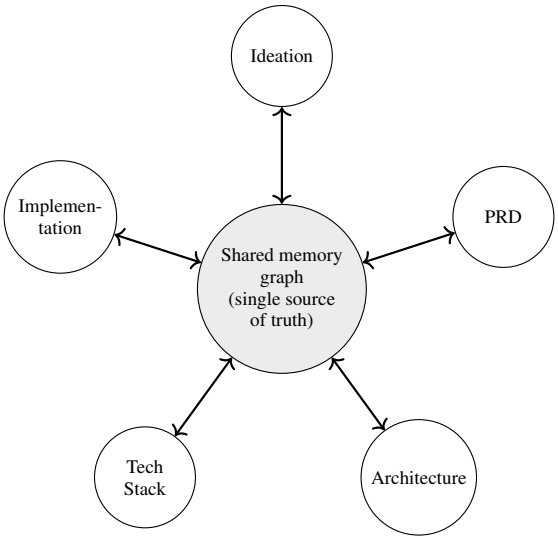


Figure 2: Stage-level structure: agents read and write the shared memory graph; they do not communicate directly.

Table 7: Reference Entity Types

Entity	Meaning
Requirement	A stated need or constraint on the system
Decision	A choice made, with rationale and alternatives
Component	A module, service or file-level unit of the design
Interface	A contract between components
CodeChange	A change recommended or integrated, with location
Test	A check and its outcome
Verification	A review result for a submitted change
Task	A unit of work requested by the developer

V.3 Hybrid Vector–Graph Storage

Each knowledge item has two representations. Its textual content is embedded in a vector space for semantic retrieval, and its identity and relations are stored in the graph. Retrieval for a query q is hybrid: a vector search proposes semantically relevant seed items, and graph expansion adds neighbours that are structurally related (for example, the interface a retrieved component depends on, or the requirement a retrieved decision was motivated by). This yields $C(q, t)$ that is both topically relevant and structurally complete.

V.4 Write Path

Knowledge enters memory in three ways: (i) the developer injects requirements, decisions and context directly; (ii) a reasoning role writes its outputs, such as an architecture description or a test plan; (iii) a verification result is recorded when the developer submits an implementation. Each write is a *mutation* $m = (\text{type}, \text{items}, \text{relations}, \text{source}, \text{time})$ that is applied to memory and published as an event (Section VI).

V.5 Persistence and History

Because mutations carry a source and a timestamp, memory retains history: an item that is replaced is linked to its successor through *supersedes* rather than silently overwritten. This lets retrieval prefer current items while still making earlier decisions and their rationale recoverable, which addresses the practical situation in which a team needs to know *why* something was changed.

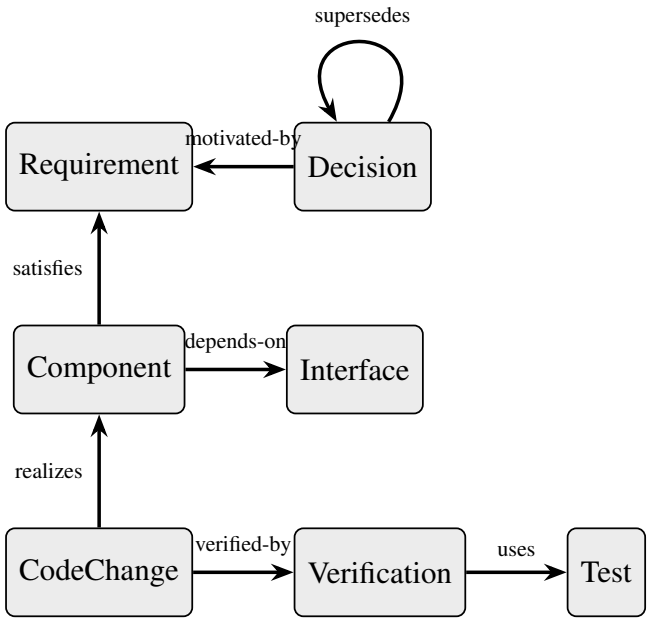


Figure 3: Reference schema of the project memory graph (entity types and principal relations).

V.6 Consistency

Roles may produce overlapping or conflicting items. We adopt a simple policy: a new item that contradicts an existing item of the same type is stored with a *supersedes* or *conflicts-with* relation instead of being discarded, so that conflict is visible on the memory canvas and can be resolved by the developer. Full automatic conflict resolution is left to future work (Section XVI).

V.7 Memory Operations

We define the memory interface by five operations.

ingest(x) Convert raw content x (text, code, a decision) into one or more items and add them with provenance.

link(u, r, v) Add a typed relation r from item u to item v.

supersede(u, u') Mark u' as the successor of u; u remains stored.

retrieve(q, B) Return a context of at most B items for query q using hybrid search.

archive(u) Exclude u from default retrieval without deleting it.

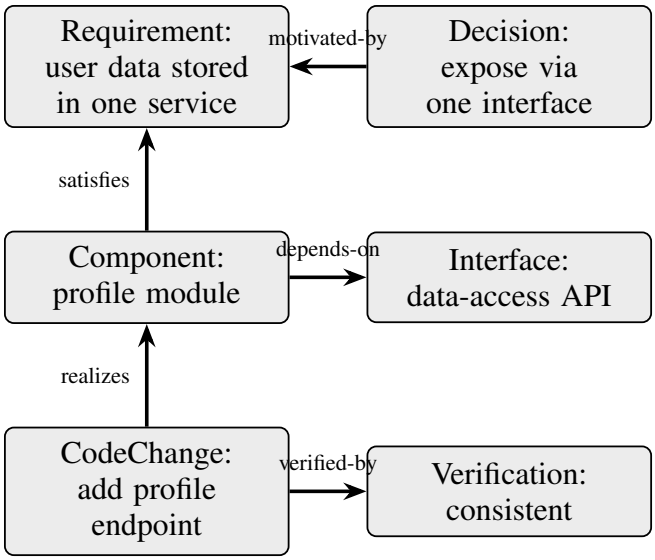


Figure 4: An illustrative instance of the memory graph (constructed example).

Every operation that changes M produces a mutation and therefore an event. `retrieve` is read-only and produces none.

V.8 Example Memory Fragment

Fig. 4 shows a small instance of the schema: a requirement motivates a decision, a component satisfies the requirement and depends on an interface, and a code change realizing the component has been verified.

V.9 Memory Quality

Memory is useful only to the extent that it is accurate and current. We identify three quality attributes. *Coverage*: the proportion of project knowledge that has been captured. *Freshness*: the proportion of retrievable items that are current, that is, not superseded or archived. *Consistency*: the absence of unresolved conflicts among current items. The canvas helps the team assess all three by making gaps, superseded items and conflict relations visible. These attributes can also be used as covariates in the evaluation (Section XIII), since a memory with poor coverage cannot be expected to give high context recall.

V.10 Memory Visualization

The memory canvas renders M as an interactive graph. Nodes are coloured by entity type, edges are labelled by relation, and selecting a node shows its content, source and history. This implements goal G6: the team can see what Syntapse "knows" and check it.

Table 8: Event Types and Subscribers (Agent Chain)

Mutation	Subscriber	Reasoning triggered
Raw prompt seeded	Ideation Specialist	Produce product vision
Vision committed or edited	PRD Architect	Produce product requirements
Requirements committed or edited	System Architect	Produce system design
System design committed or edited	Tech Stack Evaluator	Evaluate and select engineering stack
Stack committed or edited	Implementation Strategist	Produce build strategy

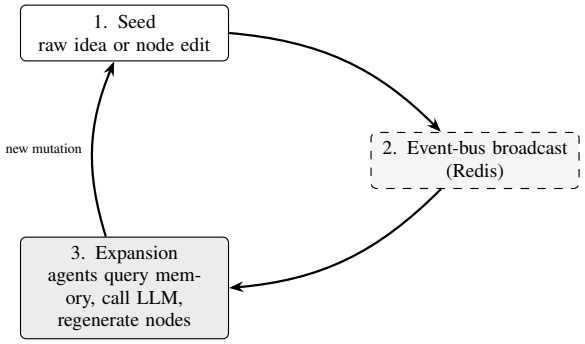


Figure 5: The self-healing event loop: seed, broadcast, expansion.

VI Event-Driven Coordination

VI.1 Memory Mutations as Events

Every mutation m applied to memory is published on the event bus (as a graph-mutated event) as an event carrying the mutation type, the affected items and the source role. Roles subscribe to the mutation types relevant to them. For instance, a role responsible for architecture subscribes to changes in requirements; a role responsible for testing subscribes to realized code changes; a role responsible for maintenance subscribes to verification outcomes and deployment records. We call the resulting behaviour *memory-mutation-driven reasoning*: the arrival of new knowledge, not an explicit prompt from the developer, is what causes dependent reasoning to run.

VI.2 The Self-Healing Event Loop

The propagation mechanism has three phases (Fig. 5). In the *seed phase*, the user enters a raw project idea or edits an existing node on the canvas. In the *event-bus broadcast* phase (the "Redis shockwave"), the event bus announces the new input or anomaly across the system. In the *expansion* phase, downstream agents wake up, query the historical context in memory, invoke a language model, and autonomously regenerate the memory nodes that are affected. The loop is self-healing in the sense that an edit or an inconsistency in one node leads to the regeneration of dependent nodes, so that the memory returns to a consistent state without the developer re-running each step.

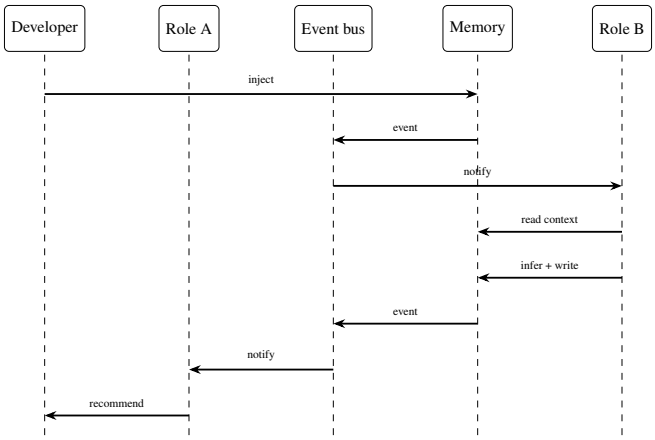


Figure 6: Propagation of a memory mutation through the event bus to dependent roles.

Algorithm 1 Memory-mutation-driven reasoning

1: **on** event e with mutation m **at** role r
2: **if** m already processed by r **or** m.source $= r$ and not revisable **then**
3: **return** {idempotence, loop guard}
4: **end if**
5: $q \leftarrow \text{formulate}(r, m)$
6: $C \leftarrow \text{retrieve}(M, q, B)$ {hybrid vector + graph}
7: $y \leftarrow \text{infer}(r, q, C)$ {local or hosted model}
8: $m' \leftarrow \text{toMutation}(y)$
9: **if** m' adds or changes knowledge **then**
10: $\text{apply}(M, m')$; $\text{publish}(m')$
11: **end if**
12: mark m processed by r

VI.3 Propagation Sequence

Fig. 6 shows the propagation of a single mutation. The developer injects knowledge; the mutation is written to memory and an event is published; a subscribed role reads the relevant context from memory, reasons using a model, and writes its result back as a new mutation, which may trigger further events.

VI.4 Algorithm

Algorithm 1 summarizes the handling of a mutation by a subscribed role.

VI.5 Idempotence, Ordering and Termination

Because an event may be delivered more than once, handlers must be idempotent; recording which mutations a role has processed (line 2) prevents duplicate work. Because roles can produce mutations that other roles consume, cycles are possible in principle. Termination is ensured by the guard on line 2 and by the rule that a mutation is published only if it adds or changes knowledge (line 8): a handler that would reproduce what memory already contains emits nothing, so propagation stops when memory reaches a fixed point. This behaviour underlies what the project describes as a self-healing loop: a failed verification produces a mutation that triggers a revised recommendation, which can be re-verified.

Table 9: Agents and Their Use of Shared Memory

Agent	Reads	Writes
Ideation Specialist	Raw prompt, prior ideas	Product vision
PRD Architect	Vision	Product requirements
System Architect	Vision, requirements	System design (components, structure)
Tech Stack Evaluator	Requirements, system design	Engineering stack choices
Implementation Strategist	All earlier nodes	Build strategy: modules, goals, steps and prompts

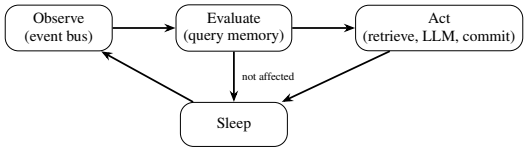


Figure 7: Lifecycle of an agent daemon.

VI.6 Failure Handling

If a model call fails or returns unusable output, the handler records a failure mutation rather than silently dropping the event, so the failure is visible and retryable. Because memory is persistent, the system can resume after a restart from the recorded state instead of from a lost conversation.

VII Specialized Agents and Their Lifecycle

VII.1 The Five Daemons

Table 9 states what each agent reads from and writes to the shared memory. All agents use the same memory.

VII.2 Agent Lifecycle: Observe, Evaluate, Act, Sleep

Each agent runs as a background daemon and follows a four-state lifecycle (Fig. 7). **Observe:** the agent listens to the Redis event bus for memory mutations. **Evaluate:** it queries Cognee to check whether its own responsibility (for example, generating the architecture plan) is missing or invalidated. **Act:** it retrieves the necessary context, invokes a language model, and commits the new state back to Cognee. **Sleep:** it returns to observation mode. This lifecycle is intended to support long-running work without continuous prompting by the developer.

Table 10: Memory Lifecycle Events

Event	Meaning
Remember	An agent commits a refined concept to the Cognee graph
Recall	An agent queries the graph for historical context
Improve	A human explicitly injects new truth into a memory node
Forget	A downstream agent discards a stale plan because its foundational memory was altered

VII.3 Memory Lifecycle: Remember, Recall, Improve, Forget

Syntapse translates backend execution into a visible "thread of consciousness" on the interface, a timeline of human-readable cognitive events with four kinds (Table 10).

VII.4 Autonomous Healing

Because Syntapse is event-driven around a memory graph, it can heal itself. If a user edits a memory node (for example, changing the technology stack from React to Rust), the system does not need to be restarted: the edited memory emits an event, and downstream agents detect the anomaly in their foundation, discard their stale plans, and regenerate new ones based on the updated state.

VII.5 Extensibility to Further Lifecycle Stages

The current agents cover the life cycle from ideation to implementation planning. The architecture is intended to extend to later stages: additional specialized agents, such as a QA daemon for unit and integration testing and bug detection, or a DevOps daemon for CI/CD, monitoring and feedback, can be attached to the memory graph as observers without altering the logic of existing agents. These agents are part of the platform vision and are not part of the evaluated prototype; secure terminal execution, which testing agents would need, is listed as future work.

VII.6 Form of Agent Outputs

To keep outputs consumable by other agents and by the developer, each agent writes output in a consistent, structured form (a JSON-like node whose content the developer can read and edit), so that mutations can be created mechanically from agent output and the canvas stays readable.

VIII Human-in-the-Loop Workflow

VIII.1 Rationale

Syntapse intentionally keeps the developer in the loop. Language models can produce plausible but wrong output, and the developer is accountable for the codebase. A design in which the system proposes, regenerates and checks while the person decides keeps responsibility with the developer; in Syntapse, developers retain control by overriding AI decisions through edits of the memory graph at any point.

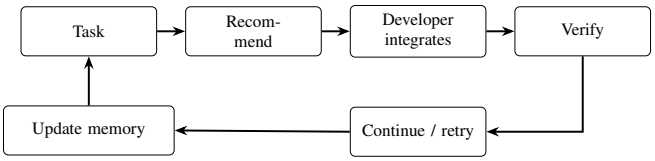


Figure 8: Iterative human-in-the-loop workflow. Each outcome is written back to shared memory.

VIII.2 Workflow

The workflow comprises two cooperating loops. The *planning loop* is autonomous: the developer enters a raw project idea, and the five agents expand it into product vision, requirements, system design, engineering stack and build strategy while the developer watches the memory canvas grow. The *implementation loop* (Fig. 8) keeps the developer in charge of integration.

1. **Seed.** The developer enters a raw project idea.
2. **Planning.** The agents write the planning nodes to shared memory (Section VI).
3. **Build strategy.** The Implementation Strategist produces a build strategy divided into modules, each with a goal, instructions and an expected output, and with a prompt that can be given to an AI coding assistant.
4. **Integration.** The developer carries out the module, using the strategy as guidance.
5. **Verification.** The developer pastes the resulting AI output or walkthrough back into Syntapse, which reviews it against the recorded context and reports problems (for example, files created but required environment values not provided).
6. **Decision.** The developer chooses to continue, retry or edit the output. The outcome is written to memory and may trigger further events.

VIII.3 Visual Steering of the Memory Graph

Beyond the verification loop, the developer steers Syntapse by editing the memory graph visually on the canvas: selecting a node opens a spatial popover in which its data can be edited, and an "Inject Memory" action commits the change. Editing a node (for example, changing a requirement in the PRD or replacing a technology choice) is itself a mutation: it is written to memory, broadcast on the event bus, and the affected downstream nodes are regenerated. The developer can therefore override an AI decision at any point, and the override becomes part of the context that every later agent uses. This is the sense in which the platform keeps a human in the loop: the AI proposes and regenerates, the developer decides.

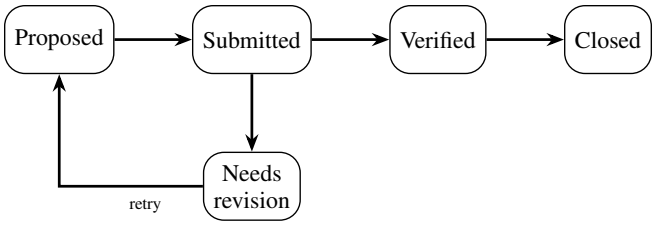


Figure 9: Task states in the verification loop.

Algorithm 2 Verification of a developer submission

Require: task t, recommended change c, submitted code x
1: $C \leftarrow \text{retrieve}(M, (t, c, x), B)$
2: $v \leftarrow \text{review}(x, c, C)$ {consistency with requirements, interfaces, prior decisions}
3: record Verification v linked to t and c in M
4: **if** v has no blocking issues **then**
5: offer **continue** (and possibly **retry** to improve)
6: **else**
7: offer **retry** with revised recommendation from v
8: **end if**
9: developer selects an option

VIII.4 State Machine

The life of a task can be described by the states in Fig. 9: *Proposed* (a recommendation has been issued), *Submitted* (the developer has submitted code), *Verified* (review found no blocking issues), *Needs revision* (review found issues) and *Closed*. Transitions are driven by developer actions and by verification results, and every transition is a mutation recorded in memory.

VIII.5 Verification Procedure

Algorithm 2 describes verification. The submitted code is compared with what was recommended and with the retrieved requirements and interfaces. The result lists issues (if any) and proposes next options. Because the review is model-assisted, its outcome is advisory and the developer decides.

VIII.6 Retry Policy and Escalation

A retry produces a revised recommendation that takes into account the earlier verification result, which is available in memory. To avoid unproductive loops, the interface shows the number of attempts for a task and the issues that remain unresolved across attempts. If the same issue persists, the developer may add clarifying knowledge to memory (for example, an omitted constraint), which triggers fresh reasoning with better context, or may close the task manually. This makes the developer's corrections durable: once a constraint has been added, it benefits all later tasks.

VIII.7 Presentation of Options

After verification, Syntapse presents the developer with explicit options rather than acting on its own: continue to the next step, retry with a revised recommendation, or refine the current approach. Presenting options keeps decisions with the developer and makes each transition in the task state machine an intentional act.

VIII.8 Implications

Keeping a human in the loop trades speed for control and accountability. The developer reviews each change before it enters the codebase, while memory ensures that each iteration builds on everything recorded so far. We therefore describe Syntapse as an *assisted* engineering platform whose coordination among lifecycle roles is automated through memory and events, not as a system that autonomously ships software.

VIII.9 Formal Properties

We state two properties of the event mechanism under explicit assumptions.

Proposition 1 (Idempotent handling)**.** *If each role records the identifiers of mutations it has processed and skips any mutation already recorded (Algorithm 1, line 2), then redelivery of an event does not cause a second application of that role's reasoning to the same mutation.*

Argument. The first delivery records the identifier after handling; any later delivery of the same mutation finds the identifier and returns before retrieval or inference. □

Proposition 2 (Termination)**.** *Assume the set of possible items is finite and a handler publishes a mutation only if it adds or changes at least one item. Then any chain of events triggered by a single mutation is finite.*

Argument. Each published mutation changes the memory state. Under the assumption, a mutation that would leave the state unchanged is not published, so a chain can continue only while the state changes. With a finite state space and a monotone growth of history, the chain cannot continue indefinitely unless the state revisits earlier states through repeated alternation (for example, two roles reversing each other's change). Such alternation is prevented by the loop guard that prevents a role from reprocessing its own non-revisable output and by the per-task limit on revisions. Hence the chain terminates. □

These properties are intentionally modest. They do not assert that the terminal memory state is correct, only that propagation stops and does not duplicate work.

VIII.10 Levels of Autonomy

Table 11 situates Syntapse on a scale of autonomy, which clarifies what the platform does and does not claim.

Syntapse automates *coordination among roles* and the regeneration of planning artifacts (idea, PRD, architecture, tech stack) at level L3, but not *integration of code*, which remains with the developer; direct repository integration is future work. The words "autonomous reasoning" used for the

Table 11: Levels of Autonomy in Engineering Assistance

Level	Description	Example
L0	Developer does everything	No assistant
L1	Assistant answers queries	Prompt-only help
L2	Assistant recommends, developer integrates	Syntapse recommendation step
L3	Assistant coordinates lifecycle roles; developer integrates and approves	Syntapse (events, verification)
L4	Assistant integrates and ships with oversight	Not claimed

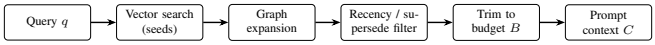


Figure 10: Context assembly pipeline from query to prompt context.

event mechanism therefore refer to the roles reacting to memory without prompts from the developer, not to unsupervised changes to the codebase.

IX Context Assembly and Model Back-ends

IX.1 Context Assembly

For each reasoning step, the role forms a query q from the triggering mutation or developer request and retrieves a context $C(q, t)$ under budget B. Assembly proceeds by (i) vector search for seed items, (ii) graph expansion along relations relevant to the role, (iii) preferring current items over superseded ones while retaining rationale links, and (iv) trimming to the budget. Because the budget is applied to a *selected* subset of a persistent memory, projects can grow beyond any model's context window without losing the ability to bring the right knowledge into a prompt.

IX.2 Retrieval Pipeline

Fig. 10 shows the retrieval pipeline used for context assembly. The pipeline is the same for every role; roles differ in the query they form, the relations they prefer during expansion, and the budget they are allowed.

IX.3 Cost Considerations

Let $N = |V|$ be the number of items in memory, k the number of seeds returned by vector search, and d the number of expansion hops. With an index for vector search, seed retrieval is sublinear in N in practice. Graph expansion visits at most $k \cdot \bar{b}^{\,d}$ items, where $\bar{b}$ is the average branching factor over the relations followed, so the cost is controlled by choosing small d (typically one or two hops) and by following only relations relevant to the role. The prompt size is bounded by B regardless of N, so inference cost per step does not grow with the size of the project, whereas an approach that places the whole project in the prompt grows linearly until it no longer fits.

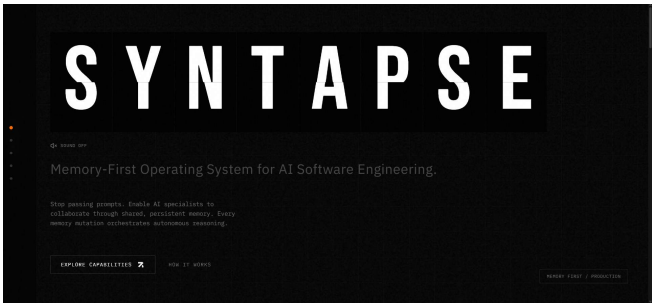


Figure 11: Landing page of the Syntapse web application.

IX.4 Local and Hosted Models

Syntapse supports local inference with Qwen2.5 through Ollama and hosted inference with Gemini. Local inference keeps code and project knowledge on the developer's infrastructure, which is relevant for confidentiality; hosted inference may offer higher capability and lower setup cost. Since project knowledge lives in memory and not in the model, switching between models does not discard project history (G7).

IX.5 Prompt Construction

Prompts are built from three parts: a role instruction (what this lifecycle role does and in what form it answers), the retrieved context, and the specific request or submission. Separating role instruction from context makes it possible to improve a role without touching memory, and to improve memory without touching roles.

X User Interface and Memory Canvas

The interface is a Next.js web application that presents the system as a spatial "OS canvas". Fig. 11 shows the landing page, which presents Syntapse as a memory-first operating system for AI software engineering. Fig. 12 shows the memory canvas after a project idea (an example project idea entered as the prompt) has been expanded: the graph grows organically from the Raw Prompt node through Product Vision, Product Requirements, System Design and Engineering Stack to the Build Strategy node as agents commit memories. Fig. 13 shows the execution plan view together with the left-hand timeline of memory events (remember, recall, improve), which makes visible that the agents are using memory and not hard-coded scripts. Fig. 14 shows the build-strategy view with its modules and the field in which the developer pastes the output of implementation work for review, together with the progress indicator of the agent currently working.

The demonstration follows four steps. In *the seed*, the developer enters a raw project idea, which appears as the initial Raw Prompt node. In *the expansion*, the developer takes their hands off the keyboard while the canvas expands as agents retrieve context and form new memories. In *the thread of*

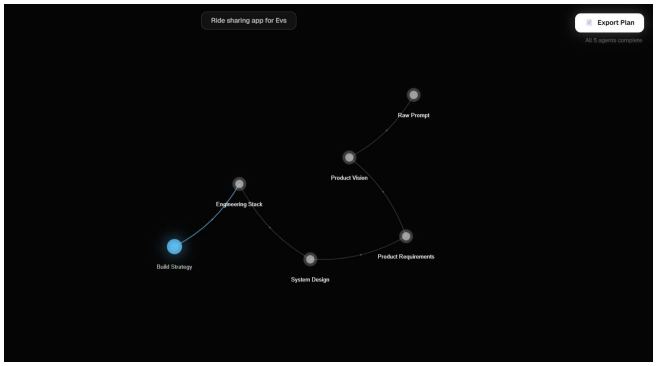


Figure 12: The 3D memory canvas growing as agents commit memories.

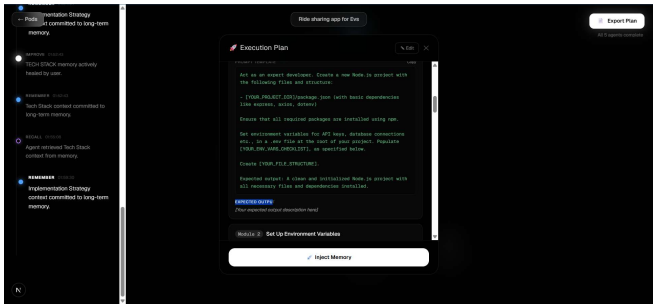


Figure 13: Execution plan view with the timeline of memory events (remember, recall, improve).

consciousness, the timeline fills with human-readable cognitive events. In *the healing*, the developer clicks a node such as the tech-stack node, edits its data to change a core technology, and injects the memory; a visual shockwave ripples through the graph, downstream nodes are discarded, and they are regenerated from the new state.

XI Implementation

XI.1 Components

The frontend is a Next.js (React) application using framer-motion for animation and react-force-graph for rendering the memory graph. The backend is a Python FastAPI service built on asyncio in which the agent daemons run. Cognee manages ingestion and retrieval over the memory graph, with Memgraph as the graph store, and Redis Pub/Sub provides the event bus; the two databases are started locally with Docker Compose. Supabase provides user authentication and basic persistence. Agents call either Ollama-served Qwen2.5-Coder (3B) or Gemini. The code base is predominantly TypeScript (about 72%) and Python (about 26%). The system originated as an entry to a hackathon on the theme of context loss in AI systems.

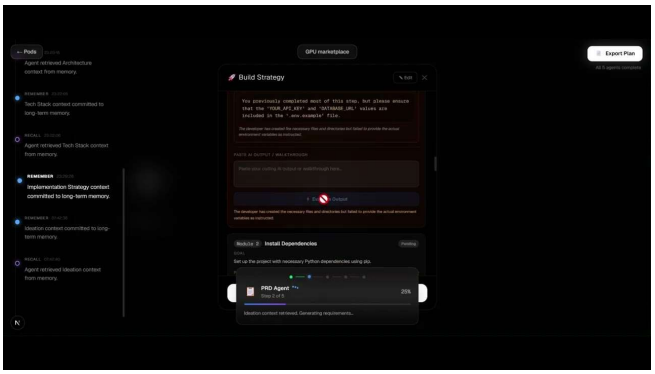


Figure 14: Build strategy: modules, submission of implementation output, and agent progress.

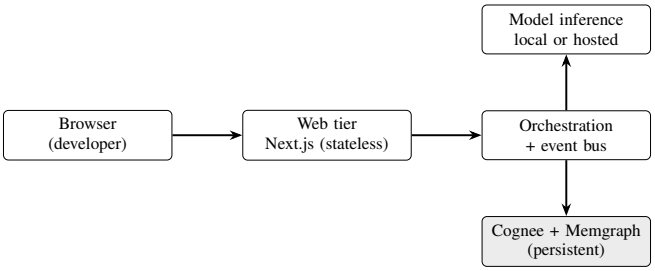


Figure 15: Deployment view: stateless web tier, persistent memory core, and interchangeable inference.

XI.2 Interaction Between Components

When a developer enters a prompt, the front end sends it to the backend, which commits a Raw Prompt node through Cognee. The memory layer publishes a graph-mutated event on Redis; subscribed daemons evaluate whether their node is missing or invalid, retrieve context, call a model and commit results. Updates are pushed to the interface over WebSockets so that the canvas and the event timeline reflect the new state in real time.

XI.3 Deployment Considerations

Memory services and the event bus are stateful and are run as services separate from the web interface. Model inference can be local to the team or remote. Because the system of record is the memory layer, the web tier can be redeployed without loss of project knowledge.

XI.4 Deployment View

Fig. 15 shows the deployment view. The web tier is stateless and can be hosted on a web platform; the stateful services (memory, graph database, event bus) form the persistent core; model inference is either co-located with the team or remote.

Table 12: Design-Level Comparison

Aspect	Session-bound assistants	Message-passing multi-agent	Syntapse
Project context	Prompt / window	Message history	Persistent graph + vector memory
Coordination	None / manual	Messages, prompt chains	Memory mutations and events
Cross-session recall	Re-supplied by user	Often per run	Retained in memory
Rationale tracing	Implicit	Implicit	Explicit relations
Verification	Varies	Often agent-internal	Developer-submitted, model-reviewed
Visibility	Opaque	Logs	Memory canvas

Table 13: Illustrative Propagation of a Requirement Change

#	Event / action	Effect
1	Developer edits the requirements node	Improve event; mutation written and published
2	System Architect notified	Discards stale system design and regenerates it from the new requirements
3	Tech Stack Evaluator notified	Re-evaluates the engineering stack for the new design
4	Implementation Strategist notified	Regenerates the build strategy
5	Developer follows the strategy and submits output	Submission reviewed against recorded context
6	Review reports an issue; developer retries and passes	Outcome recorded in memory

XI.5 Operational Concerns

Observability. Because every step is a mutation with a source and a parent event, the causal chain of any result can be reconstructed, which supports debugging and audit. **Scaling.** The web tier scales horizontally because it is stateless; the memory and event services scale according to the size of the project and the number of concurrent users; model inference is typically the dominant cost and can be bounded by limiting the budget B and the number of triggered reasoning steps. **Cost control.** The developer can see which events triggered reasoning, and roles can be disabled for a project when their output is not needed. **Backup.** Since memory is the system of record, regular backup of the memory services is the primary data-protection measure.

XI.6 Availability of Artifacts

A demonstration video (https://youtu.be/rS-OEcSBRpg) and a technical blog post (https://dev.to/yaser-123/we-stopped-passing-prompts-we-started-sharing-memory-3b3k) describe the system; instructions for running it locally are given in the project README.

XII Discussion

XII.1 Comparison with Session-Bound and Message-Passing Approaches

Table 12 summarizes design-level differences. It is a qualitative comparison of design choices, not a measured benchmark.

XII.2 Illustrative Scenario

Consider a team that has recorded a requirement that user data must be stored only in a particular service, and a decision to expose it through a specific interface. Weeks later a developer asks for a new feature that needs that data. In a prompt-bound assistant the developer must remember and restate both the requirement and the decision. In Syntapse the request triggers retrieval; the graph links the feature's component to the interface and the requirement, so the recommendation respects both without being told. If the developer's implementation violates the constraint, verification, which retrieves the same items, can flag it. This scenario is illustrative of the intended behaviour and is not a reported experiment.

XII.3 Design Alternatives Considered

Prompt-only context. Simplest, but fails goals G1 and G2 because knowledge disappears when a session ends and differs between roles. **Vector-only memory.** Gives persistence but not explicit relations, so dependencies and rationale must be rediscovered by similarity, failing G4. **Shared database without events.** Gives a shared view but requires manual or scheduled invocation of roles, failing G3. **Fully autonomous application of code.** Removes the developer from integration, which conflicts with G5 given that generated output can be wrong. The chosen combination of hybrid memory, events and developer-controlled verification is the smallest design that satisfies G1 to G7.

XII.4 Worked Example: Requirement Change Propagation

Table 13 traces how a change to a requirement propagates. The example is a constructed illustration of the mechanism and not a measured run.

Steps 2 to 4 occur without the developer invoking those agents, which is the effect of memory-mutation-driven reasoning, while steps 5 and 6 keep the developer responsible for integration. Every step leaves a trace in memory that can be inspected on the canvas.

XII.5 Comparison of Interaction Overhead

Let r be the number of roles that depend on a changed item. In a developer-orchestrated workflow the developer performs $\Theta(r)$ invocations and restates context each time. In the event-driven design the developer performs one write, and the r dependent roles are notified automatically. This does not reduce

the compute spent on reasoning; it reduces the developer's coordination effort, which RQ3 (Section XIII) is designed to measure.

XII.6 Use Cases

Onboarding. A new team member can explore the memory canvas and query Syntapse to understand what was decided and why, instead of reconstructing history from scattered documents. **Changing requirements.** A change is written once and propagates to dependent reasoning through events (Table 13). **Regression investigation.** When a defect appears, the relations from a code change to its component, requirement and decision indicate which constraints were in force and which verification was recorded. **Maintenance handover.** Operational notes and follow-up tasks are stored with the changes they concern, so handover does not depend on the memory of individuals.

XII.7 Relation to Established Practice

Several established practices address parts of the same problem: requirements traceability links requirements to design and code, decision records document why architectural choices were made, and code review checks changes against team expectations. Syntapse does not replace these practices; it provides a substrate in which their artifacts are stored as connected items and are available to an AI assistant automatically, and in which the assistant's own outputs are recorded in the same structure.

XII.8 Parameter Sensitivity

Retrieval behaviour depends on the number of seeds k, the expansion depth d and the budget B. Small k and d favour precision and low cost but may miss relevant structure; large values improve recall but dilute the prompt with marginally relevant items; the 36% empty patches reported for the hybrid context in SE-Blackboard [6] suggest that surfacing too much memory at once can harm results. We therefore treat these parameters as role-specific and as experimental variables in the evaluation rather than as fixed constants.

XII.9 Anticipated Objections

"Long-context models make memory unnecessary." Larger windows reduce but do not remove the problem. Cost grows with prompt length, and a window still ends when the session ends. Persistent memory complements long context by deciding *what* to place in it. **"A graph is overhead compared with a vector index."** The graph adds ingestion work, but it records dependencies and rationale that similarity search cannot, and it supports the memory canvas. Whether the benefit justifies the cost is an empirical question addressed by the ablation of vector-only and hybrid memory. **"Keeping a human in the loop defeats the purpose of automation."** The automation targeted here is the coordination of roles and the maintenance of context, which are tedious and error-prone for people; integration of code is deliberately left to the developer for accountability. **"Model review is not verification."** We agree that model-assisted review gives no formal guarantee; it is presented as advisory, and execution of tests is listed as future work so that verification can include objective evidence.

XII.10 Benefits

Persistence removes the burden of re-explaining. Shared memory prevents roles from diverging. Events remove manual orchestration. Graph structure supports rationale tracing. The canvas makes knowledge auditable. Human verification preserves accountability.

XII.11 Trade-offs

Memory introduces new failure modes: stale or wrong items can mislead later reasoning, and ingestion and retrieval add latency and infrastructure to operate. Event-driven propagation can trigger more reasoning than the developer needs, which costs compute. Keeping a human in the loop slows throughput relative to a fully automatic pipeline. These trade-offs are deliberate: they favour reliability and control over speed.

XII.12 Applicability

The memory-first approach is most useful for projects that last long enough for forgetting to matter, that involve several people or sessions, and where decisions and rationale are valuable. It is less compelling for very small, one-off scripts.

XIII Evaluation Methodology

This section defines how Syntapse should be evaluated. The project presentation lists intended advantages, namely retention of structural relationships and historical decisions, scalability through attachable agents, and human-in-the-loop steering; here we treat them as design intentions to be tested rather than as established results. The paper reports design and implementation; quantitative results are not claimed here.

XIII.1 Research Questions

The overall research question is: how can persistent graph-based memory and event-driven communication improve context preservation and coordination among autonomous AI agents throughout the software development life cycle? We decompose it as follows.

RQ1 Does the shared graph preserve traceability between artefacts, and does it improve context recall ρ across sessions compared with sequential prompt chaining?

RQ2 Do event notifications keep each agent's context current as artefacts change, and does change-triggered regeneration keep requirements, design and build strategy consistent?

RQ3 How well are incorrect outputs detected and corrected before downstream use (error propagation and recovery)?

RQ4 What coordination and token costs does the design add?

RQ5 Can developers inspect, edit and approve shared project memory, and at what effort (human control)?

XIII.2 Expected Outcomes and Null Results

No improvement is assumed. The evaluation is designed so that results in either direction are informative. Related work suggests plausible directions: shared state can raise information fidelity without raising final resolve rates and can cost more tokens [6]; persistent orchestration can improve execution scores at higher token cost [8]. We therefore treat benefit, no difference and added cost as all possible outcomes, and we include tasks that do not depend on prior knowledge to detect cases in which memory brings overhead without benefit.

XIII.3 Metrics

The following criteria are used. **Context retention:** whether requirements and design decisions remain available across sessions, measured by context recall ρ (Eq. 1) over queries whose relevant items are annotated in advance. **Consistency:** whether requirements, system design, stack and build strategy remain aligned after a change. **Error propagation and recovery:** whether incorrect outputs are detected and corrected before downstream use, and how many agents must regenerate work after a correction. **Software quality:** test pass rate, defect count and review findings for resulting modules. **Efficiency:** task completion time, token and API cost, and number of developer corrections. **Human control:** whether developers can inspect, edit and approve shared project memory, and the effort required. **Latency:** time from mutation to completion of the triggered reasoning.

XIII.4 Tasks and Baselines

A suite of tasks of increasing dependence on earlier project knowledge is constructed, for example changing a requirement or technology choice after the plan has been generated and checking that dependent artefacts are updated. Baselines include (a) sequential prompt chaining with the same models, (b) a shared-state design without events, in which agents read a common store but are only invoked manually, and (c) the same models with retrieval over documents and no persistent graph. Using identical models isolates the effect of the memory and event architecture from model capability, and, following the limitations noted in earlier studies, evaluation should span more than one task type and use repeated runs.

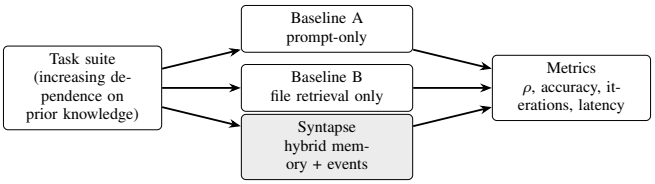


Figure 16: Evaluation design: identical tasks and models across conditions.

XIII.5 Protocol

Participants (or scripted developers) work through the task suite over multiple sessions separated in time, so that cross-session recall is exercised. Order of conditions is counterbalanced. For each task the metrics above are recorded, and for verification the reference outcome is determined independently of the system.

XIII.6 Ablations

To attribute effects to individual design elements, variants are compared: vector-only memory versus hybrid vector–graph memory; event-driven propagation versus developer-triggered invocation; and local versus hosted model.

XIII.7 Experimental Design Overview

Fig. 16 summarizes the design: the same task suite is run under each condition over multiple sessions, with identical models, and the metrics are collected per task.

XIII.8 Instruments and Sample Considerations

In addition to the quantitative metrics, perceived workload and usability can be measured with standard structured questionnaires. Short interviews can capture qualitative impressions of the memory canvas and the verification loop. Because each participant performs the task suite in several conditions, a within-subjects design reduces the number of participants needed; the sample should nevertheless be large enough to estimate effect sizes with usable confidence intervals, and a pilot study should be used to estimate variance before the main study.

XIII.9 Analysis

Paired comparisons with appropriate non-parametric tests are used where distributions are unknown, effect sizes are reported, and failure cases are analysed qualitatively.

XIII.10 Threats to Validity

Internal: learning effects across sessions; mitigated by counterbalancing. *Construct:* context recall depends on how relevant items are annotated; mitigated by independent annotation. *External:* tasks and participants may not represent in-

Table 14: Principal Risks and Mitigations

Risk	Mitigation
Disclosure of project knowledge	Access control, encryption, local inference option
Incorrect item poisoning later reasoning	Provenance, canvas inspection, supersede/archive
Untrusted text in ingested content steering the model	Label items by source; treat retrieved text as data; developer review
Unbounded event cascades	Idempotence, fixed-point rule, per-task limits
Over-reliance on model review	Verification is advisory; developer decides

Table 15: Agent Specifications

Agent	Trigger	Output	Failure mode
Ideation	Raw prompt seeded	Product vision	Vague vision
PRD	Vision committed or edited	Requirements	Ambiguity
System Architect	Requirements committed or edited	System design	Inconsistent design
Tech Stack Evaluator	Design committed or edited	Stack choice	Poor fit
Implementation Strategist	Stack committed or edited	Build strategy	Wrong module order

dustrial projects; results should be interpreted with that scope. *Model variability:* stochastic model output; mitigated by repeated runs and fixed settings where possible.

XIV Security, Privacy and Ethics

Project memory may contain confidential requirements, design details and code, so access control, encryption at rest and in transit, and per-project isolation are important. Local inference avoids sending project knowledge to a third-party service; teams using hosted inference should be aware of what context is transmitted. Memory can retain mistakes, so the ability to inspect and correct items is both a quality and an accountability feature. Finally, the human-in-the-loop design keeps responsibility for the codebase with people, which is appropriate while model output cannot be assumed correct.

XIV.1 Threat Model

Table 14 lists principal risks and mitigations.

XV Agent Specifications

Table 15 gives, for each agent, the typical trigger, the principal output and the main failure mode that the design must tolerate.

XVI Illustrative Interaction

The following constructed interaction shows the form of an implementation cycle; it is not a transcript of a recorded session.

1. The build strategy lists a module "Install dependencies" with the goal of setting up the project with the required packages and an expected output.
2. *Developer:* carries out the module with an AI coding assistant and pastes the assistant's output into the submission field.
3. *Syntapse (review):* "The required files and directories were created, but the actual values for the environment variables were not provided. Options: edit the output and resubmit, or retry the module."
4. *Developer:* supplies the missing values and resubmits; the review passes and the outcome is recorded in memory.

Each step corresponds to a mutation, and the constraint that caught the issue was retrieved from memory and not supplied by the developer in the current session.

References

[1] C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proc. ACL*, 2024. [Online]. Available: https://aclanthology.org/2024.acl-long.810/

[2] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," arXiv:2308.00352, 2023.

[3] Y. Wang *et al.*, "Agents in software engineering: Survey, landscape, and vision," arXiv:2409.09030, 2024.

[4] Z. Zhang *et al.*, "A survey on the memory mechanism of large language model-based agents," doi:10.1145/3748302.

[5] P. Du, "Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers," arXiv:2603.07670, 2026.

[6] Liu *et al.*, "SE-Blackboard," 2026. [Full title and identifier to be completed.]

[7] A. Saleh *et al.*, "MemIndex: Agentic event-based distributed memory management for multi-agent systems," doi:10.1145/3774946.

[8] W. Lyu, Y. Xiao, Y. Zhang, and Y. Sun, "Self-organizing multi-agent systems for continuous software development," arXiv:2603.25928, 2026.

[9] A. Sagirova, Y. Kuratov, and M. Burtsev, "SRMT: Shared memory for multi-agent lifelong pathfinding," arXiv:2501.13200, 2025.

[10] Chrysochos, “Society Agent,” preprint, 2026. [Full title and identifier to be completed.]